



MODELACIÓN NUMÉRICA

CURSO TARELA

Trabajo Práctico de Máquina

Misión Artemis II



12 de junio de 2026

Tomás Conti
111760

Agustín Dubovitsky Otero
111954

Índice

1. Consignas	3
1.1. Órbita Lunar	3
1.2. Trayectoria de Orion	3
2. Introducción	4
3. Desarrollo	4
3.1. Discretización Órbita Lunar	4
3.1.1. Ecuaciones Diferenciales Ordinarias	4
3.1.2. Condiciones iniciales del PVI	5
3.1.3. Elección del paso temporal	5
3.2. Discretización de la Trayectoria de Orion	6
3.2.1. Ecuaciones Diferenciales Ordinarias	6
3.2.2. Condiciones Iniciales	6
3.2.3. Elección del Paso Temporal	7
4. Resultados y Comparación de Euler y RK2	8
4.1. Esquemas de Integración	8
4.2. Estudio de la órbita lunar: Método de Euler vs. Runge-Kutta de 2do Orden (RK2)	8
4.2.1. Caso 1: Paso de tiempo pequeño ($dt = 60$ s)	8
4.2.2. Caso 2: Paso de tiempo grande ($dt = 43200$ s / 12 horas)	9
4.3. Simulación de la Trayectoria de la Nave Orion	10
4.4. Caso 2: Paso de tiempo grande ($dt = 43200$ s / 12 horas)	11
5. Conclusiones	12
6. Referencias	12
7. Códigos	12
7.1. Código pasos	13
7.2. Resolución EDOs	13
7.3. Código trayectoria Lunar	15
7.3.1. Resolución de la simulación	15
7.3.2. Gráfico de la órbita lunar	16
7.4. Código trayectoria Orion	17
7.4.1. Resolución de la simulación	17
7.4.2. Gráfico trayectoria de Orion	18
7.5. Lectura/Escritura de archivos .csv	22

1. Consignas

1.1. Órbita Lunar

La trayectoria de la nave se ve fuertemente afectada por el campo gravitatorio de la Tierra y de la Luna. Para ello, se necesita saber la posición de la Luna en todo momento. Asumiendo posición inicial y velocidad, se puede calcular la trayectoria con la siguiente ecuación diferencial:

$$\frac{d^2\vec{x}}{dt^2} = \vec{a}$$

Donde $\vec{x}$ es la posición vectorial de la Luna y $\vec{a}$ la aceleración producida por el campo gravitatorio terrestre. Si escribimos la posición y la aceleración en dos dimensiones (x e y), y descomponemos la ecuación de segundo orden en dos ecuaciones de primer orden, nos queda:

$$\frac{dx}{dt} = v_x$$

$$\frac{dy}{dt} = v_y$$

$$\frac{dv_x}{dt} = G \frac{M_T}{d_T^2} \cos \alpha$$

$$\frac{dv_y}{dt} = G \frac{M_T}{d_T^2} \sin \alpha$$

donde G es la constante de gravitación universal, M_T la masa de la Tierra, $d_T = \sqrt{x^2 + y^2}$ la distancia entre la Luna y la Tierra, $\alpha = \text{atan2}(y, x)$ el ángulo que forma el vector posición con el eje x , y h el paso temporal.

1.2. Trayectoria de Orion

Con las mismas ecuaciones, y conociendo la posición de la Luna para todo tiempo t , calcular la posición de Orion desde la fase 10 de la figura 2 hasta la fase 13. Durante todo este tiempo, la nave está (casi) en una trayectoria libre, mayormente afectada por la gravedad de la tierra y la luna. Tomar una posición y velocidad inicial aproximada, sacándolo de los datos de la telemetría, entre las 4 y las 6 am (UTC) del 3 de abril.

$$\frac{dx}{dt} = v_x$$

$$\frac{dy}{dt} = v_y$$

$$\frac{dv_x}{dt} = G \frac{M_T}{d_T^2} \cos \alpha + G \frac{M_L}{d_L^2} \cos \beta$$

$$\frac{dv_y}{dt} = G \frac{M_T}{d_T^2} \sin \alpha + G \frac{M_L}{d_L^2} \sin \beta$$

donde G es la constante de gravitación universal, M_T la masa de la Tierra, M_L la masa de la Luna, d_T la distancia entre la nave y la Tierra, d_L la distancia entre la nave y la Luna, α el ángulo que forma el vector posición con el eje x respecto a la Tierra, β el ángulo respecto a la Luna, y h el paso temporal.

2. Introducción

En este trabajo, se aborda la modelización de la trayectoria de la nave Orión en la misión Artemis II, mediante la aplicación de métodos numéricos. El objetivo principal es comparar el comportamiento observado del modelo teórico, con el de los experimentos.

La simulación se realizó con tres métodos de distinto orden, con fin de compararlos y observar el impacto que se genera en los resultados al realizar cambios en los valores iniciales.

El trabajo consta de dos partes. La primera, se centra en la obtención de la órbita de la luna sobre la tierra, y la segunda, en simular la trayectoria de orión en su recorrido entre la tierra y la luna.

3. Desarrollo

En esta sección se presentan los detalles del proceso de modelización de la trayectoria de la nave Orión, junto con los criterios adoptados en cada etapa. Ambos sistemas de ecuaciones diferenciales (la órbita lunar y la trayectoria de Orión) se resolvieron mediante el método de **Runge-Kutta de orden cuatro (RK4)**. Definiendo el vector de estado $\vec{u}_n$ en el instante t_n , el esquema de avance es:

$$\begin{aligned}\vec{q}_1 &= \vec{f}(\vec{u}_n) \\ \vec{q}_2 &= \vec{f}\left(\vec{u}_n + \frac{h}{2} \vec{q}_1\right) \\ \vec{q}_3 &= \vec{f}\left(\vec{u}_n + \frac{h}{2} \vec{q}_2\right) \\ \vec{q}_4 &= \vec{f}(\vec{u}_n + h \vec{q}_3) \\ \vec{u}_{n+1} &= \vec{u}_n + \frac{h}{6}(\vec{q}_1 + 2\vec{q}_2 + 2\vec{q}_3 + \vec{q}_4)\end{aligned}$$

donde $\vec{f}(\vec{u})$ es el campo vectorial particular de cada sistema, detallado en las subsecciones siguientes.

3.1. Discretización Órbita Lunar

Como paso previo a la simulación de la trayectoria de Orión, se modeló la órbita de la Luna alrededor de la Tierra, para conocer la posición lunar en todo instante.

3.1.1. Ecuaciones Diferenciales Ordinarias

El sistema queda descrito por las siguientes ecuaciones diferenciales ordinarias de primer orden:

$$\begin{aligned}\frac{dx}{dt} &= v_x \\ \frac{dy}{dt} &= v_y \\ \frac{dv_x}{dt} &= -G \frac{M_T}{d_T^2} \cos \alpha \\ \frac{dv_y}{dt} &= -G \frac{M_T}{d_T^2} \sin \alpha\end{aligned}$$

Las primeras dos ecuaciones las coordenadas (x, y) componentes de la velocidad. Las dos últimas corresponden a las componentes cartesianas de la aceleración gravitatoria, donde d_T es la distancia a la tierra, M_T su masa, G la constante de gravitación universal y α el ángulo que forma el vector de posición relativa con el eje x . El campo vectorial es entonces:

$$\vec{f}(\vec{u}) = \left(v_x, v_y, -G \frac{M_T}{d_T^2} \cos \alpha, -G \frac{M_T}{d_T^2} \sin \alpha \right)^\top$$

3.1.2. Condiciones iniciales del PVI

Se coloca a la Luna en el **-perigeo** como condición inicial. En ese instante el vector posición es puramente horizontal y la velocidad es perpendicular al radio:

$$x_0 = R_{\text{perigeo}}, \quad y_0 = 0, \quad v_{x,0} = 0, \quad v_{y,0} = v_{\text{perigeo}}$$

Los valores de perigeo y apogeo se obtienen de **NASA horizon system**:

$$R_{\text{perigeo}} = 356\,500 \text{ km} = 3,565 \times 10^8 \text{ m}, \quad R_{\text{apogeo}} = 406\,700 \text{ km} = 4,067 \times 10^8 \text{ m}$$

La velocidad en el perigeo se obtiene de la **ecuación de la vis-viva**:

$$v = \sqrt{GM_T \left(\frac{2}{r} - \frac{1}{a} \right)}$$

donde a es el semieje mayor de la órbita elíptica:

$$a = \frac{R_{\text{perigeo}} + R_{\text{apogeo}}}{2} = 3,816 \times 10^8 \text{ m}$$

Evaluando en $r = R_{\text{perigeo}}$ con las constantes $G = 6,674 \times 10^{-11} \text{ m}^3 \text{ kg}^{-1} \text{ s}^{-2}$ y $M_T = 5,972 \times 10^{24} \text{ kg}$:

$$v_{\text{perigeo}} = \sqrt{GM_T \left(\frac{2}{R_{\text{perigeo}}} - \frac{1}{a} \right)} \approx 1,076 \times 10^3 \text{ m/s}$$

Resumiendo, el vector de estado inicial es:

$$\mathbf{q}(0) = \begin{pmatrix} -3,565 \times 10^8 \text{ m} \\ 0 \\ 0 \\ -1,076 \times 10^3 \text{ m/s} \end{pmatrix}$$

Se utilizaron los valores en negativo para que orion encuentre a la luna, dado que si la luna empieza del otro extremo de la órbita, no va a llegar a tiempo para cuando orion pase.

3.1.3. Elección del paso temporal

Para obtener la trayectoria de una vuelta completa de la Luna en torno a la Tierra, se utilizó una duración de $T = 27,32$ días, lo cual es el tiempo oficial. El paso de tiempo h se eligió de forma que el número de pasos $N = T/h$ sea suficientemente grande para que el error de truncamiento local no acumule un error global apreciable. Se adoptó $h = 60\text{s}$, que garantiza al menos $N \approx 39,000$ puntos por órbita. Este nivel de discretización es suficiente para resolver con precisión las variaciones de posición y velocidad a lo largo de la trayectoria.

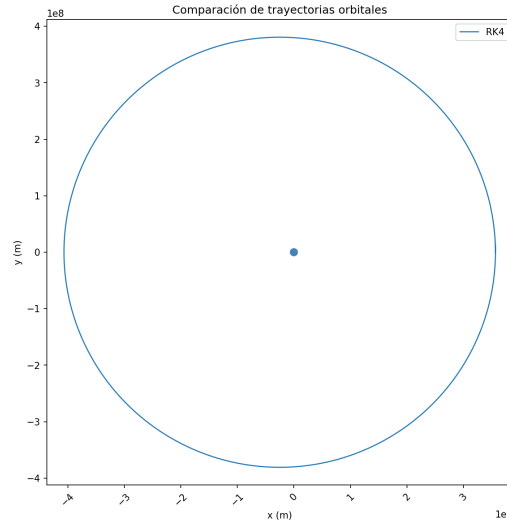


Figura 1: una vuelta exacta de la luna con el método RK4

3.2. Discretización de la Trayectoria de Orion

Habiendo calculado la órbita de la luna alrededor de la tierra, podemos calcular la trayectoria de orión para que de una vuelta por la luna y vuelva, manipulando el ángulo en el que empieza orion con fin de que se encuentre con la luna en algún momento de su trayectoria anteriormente calculada.

3.2.1. Ecuaciones Diferenciales Ordinarias

En este caso la nave está sometida a la atracción gravitatoria de tanto la luna como la tierra, por lo que el sistema de EDOs es:

$$\begin{aligned}\frac{dx}{dt} &= v_x \\ \frac{dy}{dt} &= v_y \\ \frac{dv_x}{dt} &= -G \frac{M_T}{d_T^2} \cos \alpha - G \frac{M_L}{d_L^2} \cos \beta \\ \frac{dv_y}{dt} &= -G \frac{M_T}{d_T^2} \sin \alpha - G \frac{M_L}{d_L^2} \sin \beta\end{aligned}$$

donde d_L y M_L son la distancia y la masa de la Luna, y β el ángulo que forma el vector de posición relativa a ella con el eje x . El campo vectorial correspondiente es:

$$\vec{f}(\vec{u}) = \left(v_x, v_y, -G \frac{M_T}{d_T^2} \cos \alpha - G \frac{M_L}{d_L^2} \cos \beta, -G \frac{M_T}{d_T^2} \sin \alpha - G \frac{M_L}{d_L^2} \sin \beta \right)^\top$$

3.2.2. Condiciones Iniciales

Las condiciones iniciales de posición y velocidad de la nave Orion se obtuvieron de los datos de telemetría a la hora 05:03:39 del 3 de abril. Los valores utilizados son:

$$\begin{aligned}x_0 &= -52409,924647197156 * 1000 \text{ m} \\y_0 &= -48671,635720228245 * 1000 \text{ m} \\v_{x_0} &= -1,22735334971968 * 1000 + extra_v[0] \text{ m/s} \\v_{y_0} &= -2,34643943970753 * 1000 + extra_v[1] \text{ m/s}\end{aligned}$$

$extra_v[0]$ y $extra_v[1]$ son las desviaciones realizadas sobre el ángulo de dirección de orión, para lograr que se encuentre con la luna, de la vuelta, y vuelva a la tierra.

Las condiciones iniciales del vector de velocidad resultaron de una magnitud de 3022,74 m/s y una orientación de $-127,98^\circ$.

$$\vec{v}_0 = (3022,74 \cdot \cos(-127,98^\circ), 3022,74 \cdot \sin(-127,98^\circ)) \text{ m/s}$$

3.2.3. Elección del Paso Temporal

Para la trayectoria de orion se y la órbita lunar a la hora de resolver esta trayectoria, se volvió a utilizar como paso temporal $h = 60s$, para que tengan coherencia y buena precisión.

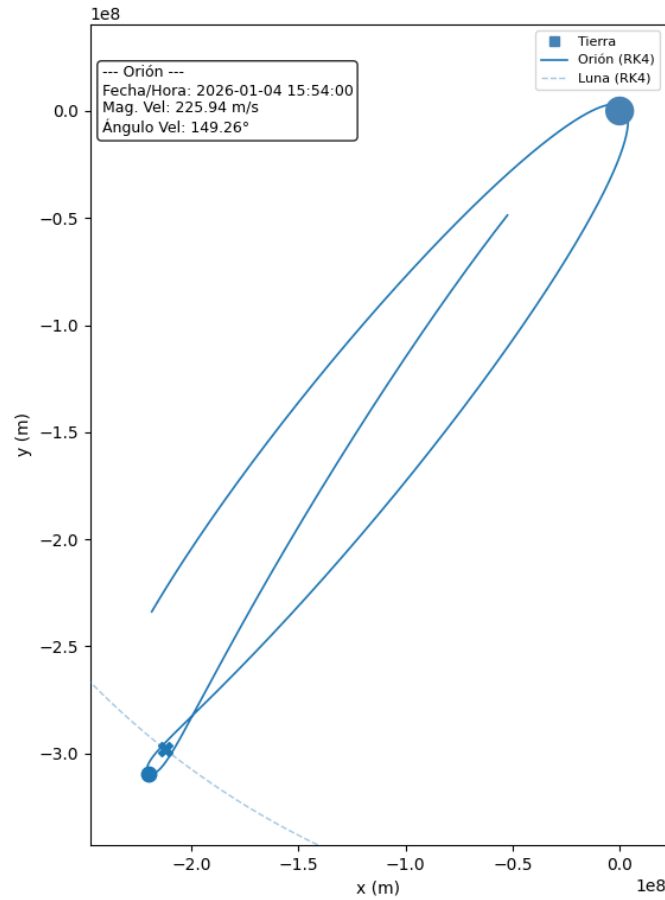


Figura 2: recorrido de orión con RK4

Se decidió utilizar un lapso de tiempo de 10 días, para asegurar que se complete el recorrido, y como podrá ver en la imagen, orion luego de dar la vuelta a la luna, volvió a la tierra y siguió, lo

cual es interesante pero para esta solución no tiene relevancia. En las próximas secciones, esto se utilizará para poder encontrar algunas diferencias entre los distintos métodos utilizados.

4. Resultados y Comparación de Euler y RK2

En esta sección se presentan y analizan los resultados de las simulaciones numéricas para la órbita de la Luna y la trayectoria de la nave Orión, calculados con los métodos de Euler explícito y Runge-Kutta de orden 2 (RK2). Para todas las simulaciones de la Luna se utilizaron como condiciones iniciales los valores correspondientes a su perigeo.

4.1. Esquemas de Integración

Euler Explícito Dado el vector de estado $\vec{u}_n$ en el instante t_n , el método de Euler explícito avanza la solución mediante:

$$\vec{u}_{n+1} = \vec{u}_n + h \vec{f}(\vec{u}_n)$$

donde h es el paso de integración y $\vec{f}(\vec{u})$ el campo vectorial del sistema.

Runge-Kutta de orden 2 (RK2) El método RK2 evalúa el campo vectorial en dos puntos del intervalo $[t_n, t_n + h]$ y los combina de forma ponderada:

$$\vec{q}_1 = \vec{f}(\vec{u}_n)$$

$$\vec{q}_2 = \vec{f}(\vec{u}_n + h \vec{q}_1)$$

$$\vec{u}_{n+1} = \vec{u}_n + \frac{h}{2} (\vec{q}_1 + \vec{q}_2)$$

En ambos métodos el campo vectorial $\vec{f}(\vec{u})$ es idéntico al utilizado en RK4. Para la órbita lunar:

$$\vec{f}(\vec{u}) = \left(v_x, v_y, -G \frac{M_T}{d_T^2} \cos \alpha, -G \frac{M_T}{d_T^2} \sin \alpha \right)^\top$$

y para la trayectoria de Orión:

$$\vec{f}(\vec{u}) = \left(v_x, v_y, -G \frac{M_T}{d_T^2} \cos \alpha - G \frac{M_L}{d_L^2} \cos \beta, -G \frac{M_T}{d_T^2} \sin \alpha - G \frac{M_L}{d_L^2} \sin \beta \right)^\top$$

4.2. Estudio de la órbita lunar: Método de Euler vs. Runge-Kutta de 2do Orden (RK2)

Con el objetivo de evaluar la estabilidad y precisión de los integradores numéricos frente al paso de tiempo (dt), se realizaron simulaciones bajo dos escenarios distintos para los métodos de Euler y RK. Para todos los casos se proyectó la órbita durante un tiempo total de 100 órbitas.

4.2.1. Caso 1: Paso de tiempo pequeño ($dt = 60$ s)

En este primer escenario se utilizó un diferencial de tiempo de 60 segundos. Los resultados de las trayectorias orbitales obtenidas se muestran en la Figura 3.

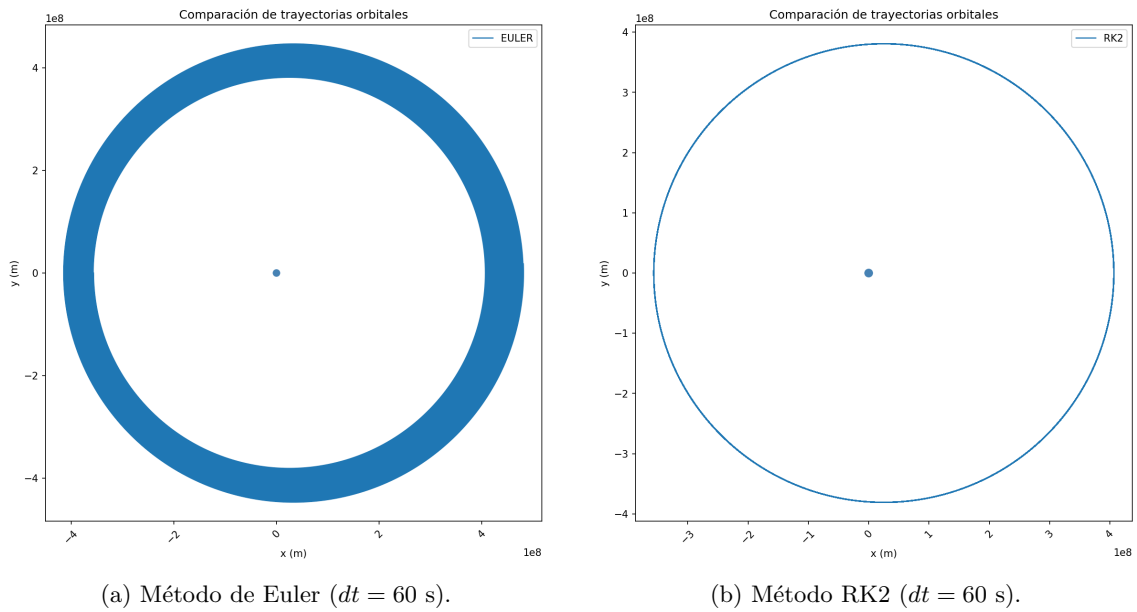


Figura 3: Comparación de trayectorias orbitales de la Luna tras 100 vueltas con un paso de tiempo corto.

Al observar la Figura 3a, se evidencia que el método de Euler presenta una acumulación de error numérica visible, manifestada como un ensanchamiento de la órbita (pérdida de conservación de energía). A pesar de utilizar un paso de tiempo pequeño, el método diverge gradualmente hacia el exterior. Por el contrario, el método RK2 (Figura 3b) muestra una estabilidad notable, manteniendo la trayectoria concéntrica y cerrada a lo largo de las 100 órbitas simuladas, demostrando una mayor precisión en el orden de convergencia.

4.2.2. Caso 2: Paso de tiempo grande ($dt = 43200$ s / 12 horas)

Para evaluar la robustez de los algoritmos, se incrementó el valor de dt a 12 horas. Los resultados se exponen en la Figura 4.

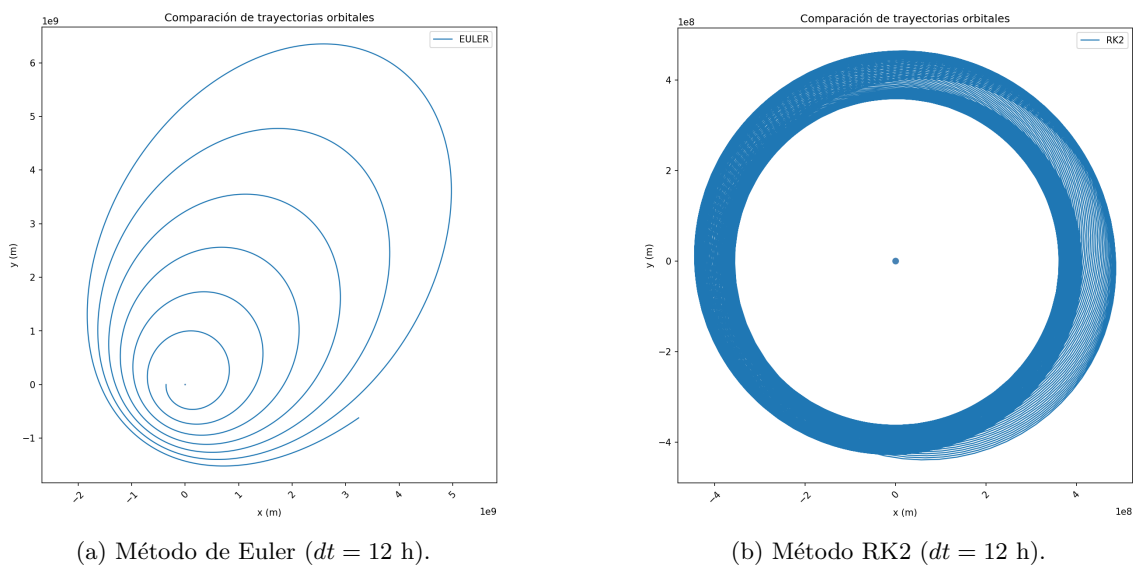


Figura 4: Comparación de trayectorias orbitales de la Luna tras 100 vueltas con un paso de tiempo de 12 horas.

Bajo estas condiciones extremas, el método de Euler (Figura 4a) colapsa por completo, describiendo una espiral divergente que aleja la Luna de su centro de gravedad de manera irreal.

Por otra parte, el método RK2 (Figura 4b) logra sostener la simulación sin divergir al infinito. Sin embargo, debido al excesivo tamaño de dt , se aprecia un fenómeno de precesión numérica que ensancha la trayectoria. Esto demuestra que, si bien RK2 es drásticamente más estable que Euler, sigue estando sujeto a restricciones de discretización temporal para preservar la física real del sistema.

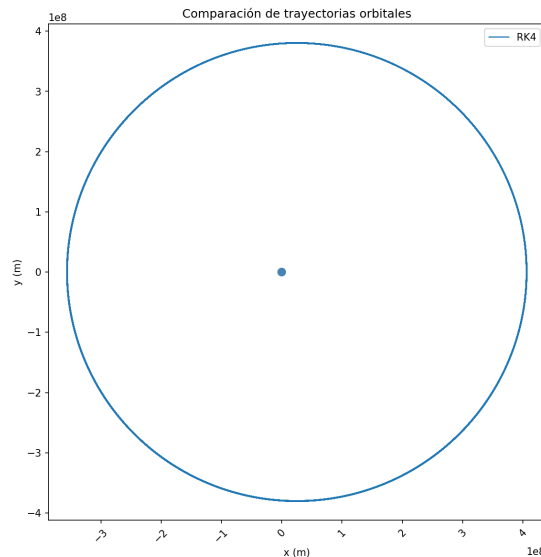


Figura 5: Método de RK4 ($dt = 12$ h).

Para el caso de la órbita lunar calculada con RK4, se puede ver que mantiene la estabilidad, incluso con un alto paso de tiempo, tras 100 vueltas, demostrando una mayor precisión.

4.3. Simulación de la Trayectoria de la Nave Orion

Habiendo determinado el comportamiento aislado del sistema Tierra-Luna, se procedió a simular la dinámica de la nave Orion interactuando bajo el campo gravitatorio de ambos cuerpos. Para asegurar que la posición del entorno de referencia fuera lo más precisa posible, la órbita de la Luna se integró utilizando un paso de tiempo óptimo de $dt = 60$ s durante un período de 1 vuelta completa.

La nave Orion se simuló durante un tiempo total de 10 días empleando el mismo diferencial de tiempo ($dt = 60$ s).

Se evaluó la misión comparando los esquemas de Euler y Runge-Kutta de 2do Orden (RK2). Los resultados se presentan en la Figura 6.

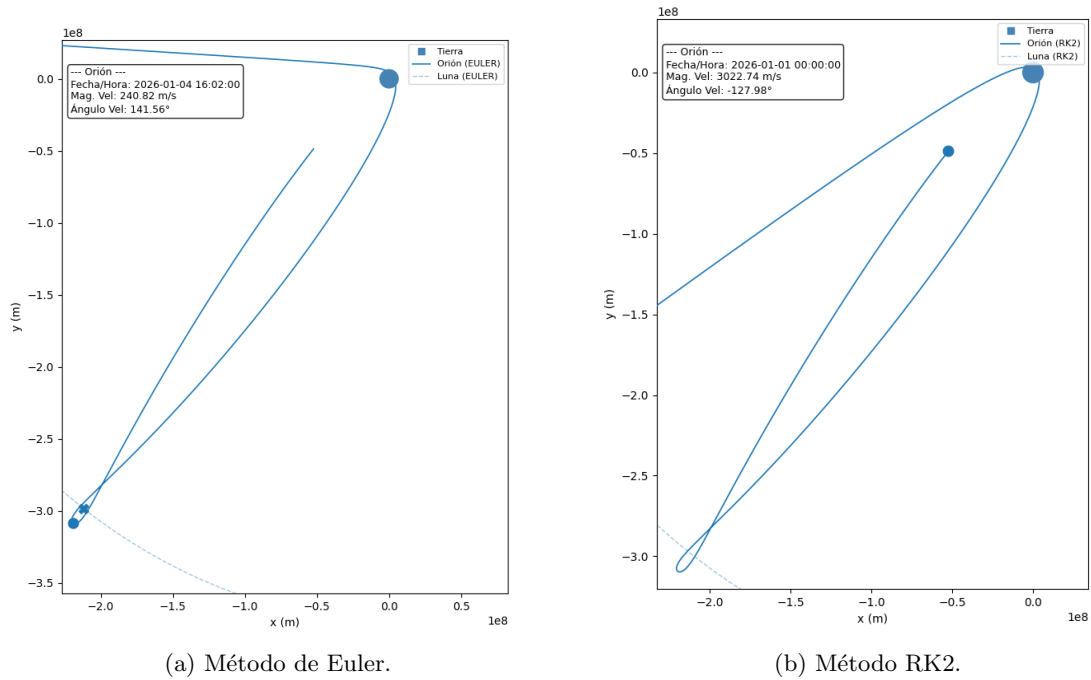


Figura 6: Trayectorias simuladas de la nave Orion en el sistema Tierra-Luna a lo largo de 10 días ($dt = 60$ s).

4.4. Caso 2: Paso de tiempo grande ($dt = 43200$ s / 12 horas)

Utilizando la órbita lunar con este paso, se calculó la trayectoria de orión con ambos métodos. Como podrá ver a continuación, con euler da un resultado totalmente erróneo, y con runge-kutta 2, la solución tampoco es correcta, pero se puede notar el contraste que tiene con euler, demostrando que con tan solo subirle un orden al método, el error de truncamiento disminuye a gran escala:

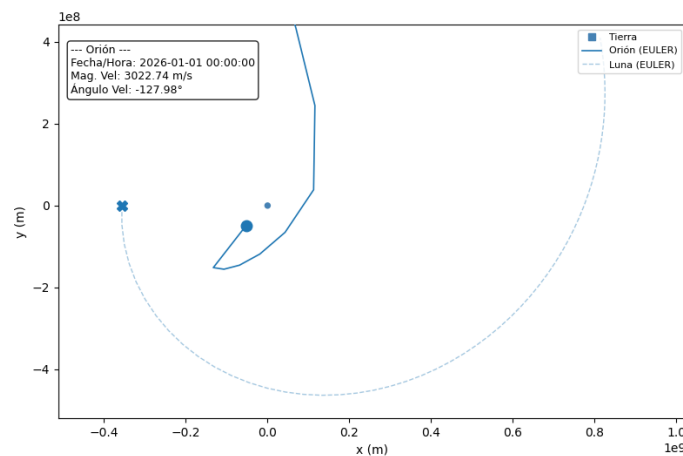


Figura 7: Trayectoria simulada de la nave Orion con el método de Euler ($dt = 43200$ s).

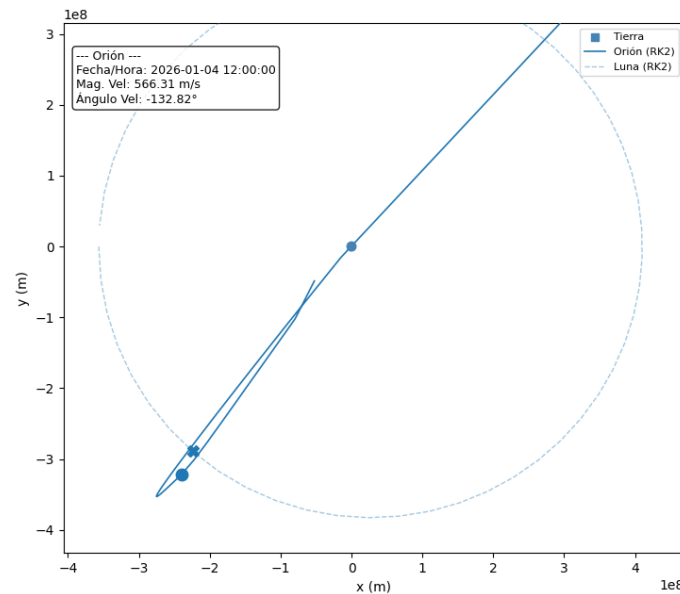


Figura 8: Trayectoria simulada de la nave Orion con el método RK2 ($dt = 43200$ s).

5. Conclusiones

Luego de haber realizado el experimento con distintos métodos, se logró entender a mayor profundidad el efecto del error de truncamiento en los métodos para PVI. Este error viene en conjunto con los valores iniciales utilizados, principalmente el valor del paso y el tiempo de duración, donde dependiendo del método, cuanto más se agrandan los valores, más se alejan de la solución real”, tal y como se vió en clase.

Por otro lado, resultó muy interesante encontrar el ángulo de velocidad inicial de orión de forma tal que la nave se encuentre con la luna y realice el recorrido correcto, ya que se pudieron observar muchos factores que afectaban la trayectoria de la nave, ya sea la fuerza que ejerce la Luna y la Tierra, lo que sucede si Orion chocaba con alguno de estos cuerpos, o si simplemente Orion no se encontraba con la Luna o a la vuelta con la Tierra.

Un método alternativo para resolver este trabajo, el cuál mejore el comportamiento en relación a Runge-Kutta 2 y Euler Explícito podría ser el método conservativo de Nystrom de tres etapas, el cuál es un método de orden 4.

Trabajar sobre la misión Artemis II resultó en una buena experiencia debido a que fue un hecho reciente e interesante, del cuál resultó un aprendizaje más allá del análisis numérico.

6. Referencias

NASA Horizon System: <https://ssd.jpl.nasa.gov/horizons>

7. Códigos

A continuación se podrán ver los códigos realizados para poder lograr la simulación de la órbita lunar y de la trayectoria de orión.

7.1. Código pasos

Los pasos que se deben hacer en cada método utilizado.

```
1 # Codigo hecho por: Agustin Dubovitsky Otero (padron: 111954) y Tomas Bautista
  Conti (padron: 111760)
2 def paso_euler(f, t, estado, h):
3     return estado + h * f(t, estado)
4
5
6 def paso_rk4(f, t, estado, h):
7     q1 = f(t, estado)
8     q2 = f(t + h/2, estado + h/2 * q1)
9     q3 = f(t + h/2, estado + h/2 * q2)
10    q4 = f(t + h, estado + h * q3)
11    return estado + (h / 6) * (q1 + 2*q2 + 2*q3 + q4)
12
13
14 def paso_rk2(f, t, estado, h):
15     q1 = f(t, estado)
16     q2 = f(t + h/2, estado + h/2 * q1)
17     return estado + h * q2
18
19
20 INTEGRADORES = {
21     'euler': paso_euler,
22     'rk2': paso_rk2,
23     'rk4': paso_rk4,
24 }
```

7.2. Resolución EDOs

Este script es de suma importancia, ya que contiene los valores iniciales que se utilizan para la resolución de las EDOs, y es el algoritmo en el cuál se resuelven las ecuaciones diferenciales normales.

```
1 # Codigo hecho por: Agustin Dubovitsky Otero (padron: 111954) y Tomas Bautista
  Conti (padron: 111760)
2 import numpy as np
3 import pandas as pd
4
5 from src.utils.pasos import INTEGRADORES
6
7 # Constantes
8
9 G = 6.674e-11
10 M_TIERRA = 5.972e24
11 M_LUNA = 7.342e22
12 RADIO_TIERRA_KM = 6371
13 RADIO_LUNA_KM = 1737
14
15 # Orbita lunar (perigeo/apogeo reales)
16 R_PERIGEO = 3.565e8
17 R_APOGEO = 4.067e8
18 A_SEMIEJE = (R_PERIGEO + R_APOGEO) / 2
19 V_PERIGEO = np.sqrt(G * M_TIERRA * (2 / R_PERIGEO - 1 / A_SEMIEJE))
20
21 # Condiciones iniciales de Orion (posicion en m, velocidad en m/s)
22 ORION_X0 = -52409.924647197156 * 1_000
23 ORION_Y0 = -48671.635720228245 * 1_000
24 ORION_VX0 = -1.22735334971968 * 1_000
25 ORION_VY0 = -2.34643943970753 * 1_000
26
27 # Dinamica Luna (solo Tierra)
```

```
28 def derivadas_luna(t, estado):
29     x, y, vx, vy = estado
30     d = np.hypot(x, y)
31     a = -G * M_TIERRA / d**2
32     return np.array([vx, vy, a * x / d, a * y / d])
33
34
35 #         Dinamica Orion (Tierra + Luna)
36
37 def _aceleracion_orion(x, y, pos_luna):
38     # Tierra
39     r_t = np.hypot(x, y)
40     a_t = -G * M_TIERRA / r_t**2
41     ax = a_t * x / r_t
42     ay = a_t * y / r_t
43
44     # Luna
45     dx, dy = x - pos_luna[0], y - pos_luna[1]
46     r_l = np.hypot(dx, dy)
47     a_l = -G * M_LUNA / r_l**2
48     ax += a_l * dx / r_l
49     ay += a_l * dy / r_l
50
51     return ax, ay
52
53 def derivadas_orion(estado, pos_luna):
54     x, y, vx, vy = estado
55     ax, ay = _aceleracion_orion(x, y, pos_luna)
56     return np.array([vx, vy, ax, ay])
57
58
59 #         Simulaciones
60
61 def simular_luna(metodo: str, dt: float, duracion_dias: float) -> pd.DataFrame:
62     integrador = INTEGRADORES[metodo]
63     pasos = int(duracion_dias * 86_400 / dt)
64     estado = np.array([-R_PERIGEIO, 0.0, 0.0, -V_PERIGEIO])
65     t = 0.0
66
67     registros = np.empty((pasos + 1, 5))
68     registros[0] = [t, *estado]
69
70     print(f" [{metodo.upper()}] dt={dt}s | {duracion_dias} dias | {pasos:,} pasos ...", end=" ")
71
72     for i in range(pasos):
73         estado = integrador(derivadas_luna, t, estado, dt)
74         t += dt
75         registros[i + 1] = [t, *estado]
76
77     print(f"listo ({pasos + 1:,} filas)")
78
79     return pd.DataFrame(registros, columns=["time_s", "x", "y", "vx", "vy"])
80
81
82 def simular_orion(metodo: str, dt: float, duracion_dias: float, estado_inicial: np.
83 ndarray, posiciones_luna: np.ndarray) -> pd.DataFrame:
84     integrador = INTEGRADORES[metodo]
85     pasos = int(duracion_dias * 86_400 / dt)
86     n_luna = len(posiciones_luna)
87     estado = estado_inicial.copy()
88     t = 0.0
89
90     registros = np.empty((pasos + 1, 5))
91     registros[0] = [t, *estado]
```

```
92     estado = integrador(f, t, estado, dt)
93     t += dt
94     registros[i + 1] = [t, *estado]
95
96     return pd.DataFrame(registros, columns=["time_s", "x", "y", "vx", "vy"])
```

7.3. Código trayectoria Lunar

7.3.1. Resolución de la simulación

Este código sirve para poder ejecutar la simulación de la órbita lunar. Al finalizar, los datos que se obtienen de la simulación, por un lado los manda a guardar en un archivo .csv, y por el otro, manda a graficar los métodos simulados. Este es el archivo principal que debe correr el usuario para poder obtener los resultados de la órbita lunar.

```
1  #!/usr/bin/env python3
2  """
3  # Código hecho por: Agustin Dubovitsky Otero (padron: 111954) y Tomas Bautista
   Conti (padron: 111760)
4  Uso:
5      python trayectoria_luna.py --metodos rk4
6      python trayectoria_luna.py --metodos rk4 euler rk2
7      python trayectoria_luna.py --metodos rk4 euler --dt 60 --duracion 27.3
8      python trayectoria_luna.py --metodos rk4 euler --salida comparacion.png
9
10     dt      : paso de tiempo en segundos (default: 60)
11     duracion : duracion de la simulacion en dias (default: 27.3)
12 """
13
14 import argparse
15 import os
16 import sys
17
18 from src.utils.pasos import INTEGRADORES
19 from utils.fisica import simular_luna
20 from graficos.io_datos import cargar_csv_trayectoria, guardar_csv
21 from graficos.graficos_luna import graficar_orbitas
22
23
24 def resolver_fuentes(metodos, dt, duracion, exportar_csv):
25     """Devuelve lista de (etiqueta, DataFrame) para cada metodo/CSV."""
26     fuentes = []
27     for m in metodos:
28         if m.startswith("csv:"):
29             ruta = m[4:]
30             if not os.path.exists(ruta):
31                 print(f"Error: no se encontro '{ruta}'", file=sys.stderr)
32                 sys.exit(1)
33             df = cargar_csv_trayectoria(ruta)
34             etq = os.path.splitext(os.path.basename(ruta))[0]
35
36             elif m in INTEGRADORES:
37                 df = simular_luna(m, dt, duracion)
38                 etq = m.upper()
39                 if exportar_csv:
40                     guardar_csv(df, f"resultados/resultado_{m.lower()}.csv")
41
42             else:
43                 print(f"Error: metodo desconocido '{m}'. Opciones: rk4, euler, rk2, csv
44                 :<ruta>",
45                       file=sys.stderr)
46                 sys.exit(1)
47
48             fuentes.append((etq, df))
49     return fuentes
50
51 def parse_args():
```

```

52 p = argparse.ArgumentParser(
53     description="Compara trayectorias orbitales lunares.",
54     formatter_class=argparse.RawDescriptionHelpFormatter,
55     epilog=__doc__,
56 )
57 p.add_argument("--metodos", nargs="+", required=True, metavar="METODO",
58     help="Uno o mas metodos: rk4, euler, rk2, csv:<ruta>")
59 p.add_argument("--dt", type=float, default=60.0,
60     help="Paso de tiempo en segundos (default: 60)")
61 p.add_argument("--duracion", type=float, default=27.3,
62     help="Duracion en dias (default: 27.3)")
63 p.add_argument("--salida", type=str,
64     default="resultados/comparacion_orbital.png",
65     help="Ruta de la imagen de salida")
66 p.add_argument("--dpi", type=int, default=150,
67     help="Resolucion de la imagen (default: 150)")
68 p.add_argument("--no-exportar-csv", dest="exportar_csv",
69     action="store_false",
70     help="Desactiva la exportacion de CSV (activada por defecto)")
71 return p.parse_args()
72
73
74 def main():
75     args = parse_args()
76     fuentes = resolver_fuentes(args.metodos, args.dt, args.duracion,
77     args.exportar_csv)
78     graficar_orbitas(fuentes, salida=args.salida, dpi=args.dpi)
79
80
81 if __name__ == "__main__":
82     main()

```

7.3.2. Gráfico de la órbita lunar

Cómo se mencionó en el apartado anterior, éste es el código que se encarga de realizar el gráfico de las trayectorias simuladas. Grafica todas sobre una imagen, utilizando distintos colores para que se pueda distinguir el comportamiento entre métodos.

```

1 #Codigo hecho por: Agustin Dubovitsky Otero (padron: 111954) y Tomas Bautista
  Conti (padron: 111760)
2 import numpy as np
3 import pandas as pd
4 import matplotlib
5 matplotlib.use("Agg")
6 import matplotlib.pyplot as plt
7 from matplotlib.patches import Circle
8
9 from utils.fisica import RADIO_TIERRA_KM
10
11 PALETA = [
12     "#1f77b4",
13     "#d62728",
14     "#2ca02c",
15     "#9467bd",
16     "#8c564b",
17     "#e377c2",
18 ]
19
20
21 def graficar_orbitas(fuentes: list[tuple[str, pd.DataFrame]],
22     salida: str,
23     dpi: int = 150) -> None:
24     """Genera y guarda una imagen comparando las trayectorias recibidas
25
26     fuentes : lista de (etiqueta, DataFrame con columnas x, y)
27     salida  : ruta del archivo de imagen resultante
28     dpi     : resolucion de la imagen
29     """

```



```

30 fig, ax = plt.subplots(figsize=(8, 8))
31
32 # Tierra
33 ax.add_patch(Circle((0, 0), RADIO_TIERRA_KM * 1_000,
34                  color="steelblue", zorder=5))
35
36 # Trayectorias
37 for (etq, df), color in zip(fuentes, PALETA):
38     ax.plot(df["x"].values, df["y"].values,
39           lw=1.2, color=color, label=etq)
40
41 all_x = np.concatenate([df["x"].values for _, df in fuentes])
42 all_y = np.concatenate([df["y"].values for _, df in fuentes])
43 cx = (all_x.min() + all_x.max()) / 2
44 cy = (all_y.min() + all_y.max()) / 2
45 hs = max(all_x.max() - all_x.min(), all_y.max() - all_y.min()) / 2 * 1.08
46 ax.set_xlim(cx - hs, cx + hs)
47 ax.set_ylim(cy - hs, cy + hs)
48 ax.set_aspect("equal", "box")
49
50 ax.set_xlabel("x (m)")
51 ax.set_ylabel("y (m)")
52 ax.set_title("Comparacion de trayectorias orbitales")
53 ax.tick_params(axis="x", rotation=45)
54 ax.legend()
55
56 fig.tight_layout()
57 fig.savefig(salida, dpi=dpi, bbox_inches="tight")
58 print(f"\n[graficos_luna] Imagen guardada: {salida}")
59 plt.close(fig)

```

7.4. Código trayectoria Orion

7.4.1. Resolución de la simulación

Este código sirve para poder ejecutar la simulación de la trayectoria de orión. Primero pide los datos obtenidos de la órbita lunar del método que se quiere ejecutar, manda a resolver el sistema de ecuaciones, y al finalizar, los datos que se obtienen de la simulación, los manda a graficar de forma interactiva. Este es el archivo principal que debe correr el usuario para poder obtener los resultados de la trayectoria de orión.

```

1 #!/usr/bin/env python3
2 """
3 # Codigo hecho por: Agustin Dubovitsky Otero (padron: 111954) y Tomas Bautista
4   Conti (padron: 111760)
5 Requiere que exista el CSV de orbita lunar correspondiente al metodo elegido
6 (generado previamente con trayectoria_luna.py).
7
8 Uso:
9     python trayectoria_orion.py --metodo rk4
10    python trayectoria_orion.py --metodo euler --dt 60 --duracion 10
11 """
12 import argparse
13 import os
14 import sys
15
16 import numpy as np
17 from PyQt5.QtWidgets import QApplication
18
19 from src.utils.pasos import INTEGRADORES
20 from utils.fisica import simular_orion, ORION_X0, ORION_Y0, ORION_VX0, ORION_VY0
21 from graficos.io_datos import cargar_posiciones_luna
22 from graficos.graficos_orion import VentanaOrbital
23
24
25 def construir_calcular_fuentes(metodo, dt, duracion) -> callable:

```

```
26     """Devuelve una funcion callable(ev0, ev1)"""
27
28     ruta_luna = f"resultados/resultado_{metodo.lower()}.csv"
29     if not os.path.exists(ruta_luna):
30         print(
31             f"Error: no se encontro '{ruta_luna}'.\n"
32             f"Ejecuta primero: python trayectoria_luna.py --metodos {metodo}",
33             file=sys.stderr,
34         )
35         sys.exit(1)
36
37     posiciones_luna, df_luna = cargar_posiciones_luna(ruta_luna)
38
39     def calcular_fuentes(ev0: float, ev1: float):
40         estado_inicial = np.array([
41             ORION_X0, ORION_Y0,
42             ORION_VX0 + ev0,
43             ORION_VY0 + ev1,
44         ])
45         df_orion = simular_orion(metodo, dt, duracion, estado_inicial,
46                                 posiciones_luna)
47         return [(metodo.upper(), df_orion, df_luna)]
48
49     return calcular_fuentes
50
51
52 def parse_args():
53     p = argparse.ArgumentParser(
54         description="Simulador interactivo de la trayectoria de Orion.",
55         epilog=__doc__,
56     )
57     p.add_argument("--metodo", required=True,
58                   help="Metodo de integracion: rk4, euler, rk2")
59     p.add_argument("--dt", type=float, default=60.0,
60                   help="Paso de tiempo en segundos (default: 60)")
61     p.add_argument("--duracion", type=float, default=10.0,
62                   help="Duracion en dias (default: 10)")
63     return p.parse_args()
64
65
66 def main():
67     args = parse_args()
68
69     if args.metodo not in INTEGRADORES:
70         print(f"Error: metodo '{args.metodo}' no reconocido. "
71             f"Opciones: {'', '.join(INTEGRADORES)}",
72             file=sys.stderr)
73         sys.exit(1)
74
75     calcular_fuentes = construir_calcular_fuentes(
76         args.metodo, args.dt, args.duracion
77     )
78
79     app = QApplication(sys.argv)
80     ventana = VentanaOrbital(calcular_fuentes)
81     ventana.show()
82     sys.exit(app.exec_())
83
84
85 if __name__ == "__main__":
86     main()
```

7.4.2. Gráfico trayectoria de Orion

Esta porción de código realiza los gráficos de la trayectoria de orión. A diferencia con la forma en la que se grafica la órbita lunar, esta se hace de forma interactiva, donde se puede simular con un slider el recorrido de orión y de la luna, donde se puede observar en que momento se encuentran. Se realizó de esta forma debido a que en este gráfico se pueden modificar las velocidades

iniciales de ori3n, de forma tal que se pueda observar interactivamente c3mo cambia el recorrido de la nave y que sucede a partir de distintos 3ngulos y velocidades iniciales. Esto fu3 de mucha utilidad para encontrar el 3ngulo inicial correcto de forma tal que orion encuentre el camino buscado. Adem3s, muestra los datos de la nave en cada instante (fecha y hora, 3ngulo de velocidad, velocidad vectorial, etc.)

```
1 # Codigo hecho por: Agustin Dubovitsky Otero (padron: 111954) y Tomas Bautista
  Conti (padron: 111760)
2 import datetime
3
4 import numpy as np
5 import pandas as pd
6
7 import matplotlib
8 matplotlib.use("Qt5Agg")
9 import matplotlib.pyplot as plt
10 from matplotlib.patches import Circle
11 from matplotlib.backends.backend_qt5agg import FigureCanvasQTagg as FigureCanvas
12 from matplotlib.backends.backend_qt5agg import NavigationToolbar2QT as
  NavigationToolbar
13
14 from PyQt5.QtWidgets import (
15     QApplication, QMainWindow, QWidget,
16     QVBoxLayout, QHBoxLayout,
17     QLabel, QLineEdit, QPushButton,
18     QSlider, QMessageBox,
19 )
20 from PyQt5.QtCore import Qt
21
22 from utils.fisica import RADIO_TIERRA_KM
23
24 PALETA = ["#1f77b4", "#d62728", "#2ca02c", "#9467bd", "#8c564b", "#e377c2"]
25
26 FECHA_INICIO = datetime.datetime(2026, 1, 1)
27
28
29 class VentanaOrbital(QMainWindow):
30     """Ventana principal del simulador de Orion.
31
32     calcular_fuentes : callable(ev0, ev1) -> list[(etq, df_orion, df_luna)]
33     Funcion que recibe los deltas de velocidad inicial y devuelve las fuentes
34     ya simuladas
35     """
36
37     def __init__(self, calcular_fuentes):
38         super().__init__()
39         self._calcular_fuentes = calcular_fuentes
40         self._fuentes = []
41         self._marcadores = []
42
43         self.setWindowTitle("Simulador Orbital Interactivo (Zoom Habilitado)")
44         self.resize(950, 850)
45
46         # Layout principal
47
48         widget_central = QWidget()
49         layout_principal = QVBoxLayout(widget_central)
50         self.setCentralWidget(widget_central)
51
52         # Controles de velocidad inicial
53         layout_controles = QHBoxLayout()
54         layout_controles.addWidget(QLabel("extra_v[0] (vx, m/s):"))
55         self._input_v0 = QLineEdit("-633")
56         self._input_v0.setFixedWidth(80)
57         layout_controles.addWidget(self._input_v0)
58
59         layout_controles.addWidget(QLabel("extra_v[1] (vy, m/s):"))
60         self._input_v1 = QLineEdit("-36")
61         self._input_v1.setFixedWidth(80)
```

```
60     layout_controles.addWidget(self._input_v1)
61
62     btn = QPushButton("Recalcular")
63     btn.setStyleSheet("background-color: #007ACC; color: white; font-weight:
bold; padding: 5px;")
64     btn.clicked.connect(self._recalcular)
65     layout_controles.addWidget(btn)
66     layout_controles.addStretch()
67     layout_principal.addLayout(layout_controles)
68
69     # Canvas de matplotlib
70     self._fig, self._ax = plt.subplots(figsize=(7, 7))
71     self._canvas = FigureCanvas(self._fig)
72     self._toolbar = NavigationToolbar(self._canvas, self)
73     layout_principal.addWidget(self._toolbar)
74     layout_principal.addWidget(self._canvas)
75
76     # Slider de tiempo
77     layout_slider = QHBoxLayout()
78     self._label_paso = QLabel("Paso: 0 / 0")
79     self._label_paso.setFixedWidth(150)
80     layout_slider.addWidget(self._label_paso)
81
82     self._slider = QSlider(Qt.Horizontal)
83     self._slider.setMinimum(0)
84     self._slider.setMaximum(100)
85     self._slider.setEnabled(False)
86     self._slider.valueChanged.connect(self._actualizar_marcador)
87     layout_slider.addWidget(self._slider)
88     layout_principal.addLayout(layout_slider)
89
90     self._texto_info = None
91     self._recalcular()
92
93     # Slots
94
95     def _recalcular(self):
96         try:
97             ev0 = float(self._input_v0.text())
98             ev1 = float(self._input_v1.text())
99         except ValueError:
100             QMessageBox.critical(self, "Error de formato", "Introduci numeros
decimales validos.")
101             return
102
103         self._fuentes = self._calcular_fuentes(ev0, ev1)
104
105         if not self._fuentes:
106             self._ax.clear()
107             self._ax.text(0.5, 0.5, "Sin datos validos.", ha="center", va="center")
108             self._canvas.draw()
109             self._slider.setEnabled(False)
110             return
111
112         max_pasos = max(len(df) for _, df, _ in self._fuentes) - 1
113         self._slider.setMaximum(max_pasos)
114         self._slider.setValue(0)
115         self._slider.setEnabled(True)
116         self._dibujar_base()
117
118     def _dibujar_base(self):
119         self._ax.clear()
120         self._marcadores.clear()
121
122         # Tierra
123         self._ax.add_patch(Circle((0, 0), RADIO_TIERRA_KM * 1_000, color="steelblue
", zorder=5))
124         self._ax.plot([], [], "s", color="steelblue", label="Tierra")
```

```
125     for (etq, df_orion, df_luna), color in zip(self._fuentes, PALETA):
126         # Trayectoria de Orion
127         self._ax.plot(df_orion["x"].values, df_orion["y"].values,
128                     lw=1.2, color=color, label=f"Orion ({etq})", zorder=4)
129         # Trayectoria de la Luna (si esta disponible)
130         if df_luna is not None:
131             self._ax.plot(df_luna["x"].values, df_luna["y"].values,
132                         lw=1.0, color=color, ls="--", alpha=0.4,
133                         label=f"Luna ({etq})", zorder=3)
134
135         marcador_orion, = self._ax.plot([], [], "o", color=color, ms=9, zorder
136 =8)
137         marcador_luna = None
138         if df_luna is not None:
139             marcador_luna, = self._ax.plot([], [], "X", color=color, ms=8,
140 zorder=8)
141
142         self._marcadores.append((df_orion, df_luna, marcador_orion,
143 marcador_luna))
144
145         all_x = np.concatenate([df["x"].values for _, df, _ in self._fuentes])
146         all_y = np.concatenate([df["y"].values for _, df, _ in self._fuentes])
147         for _, _, df_luna in self._fuentes:
148             if df_luna is not None:
149                 all_x = np.concatenate([all_x, df_luna["x"].values])
150                 all_y = np.concatenate([all_y, df_luna["y"].values])
151
152         cx = (all_x.min() + all_x.max()) / 2
153         cy = (all_y.min() + all_y.max()) / 2
154         hs = max(all_x.max() - all_x.min(), all_y.max() - all_y.min()) / 2 * 1.08
155         self._ax.set_xlim(cx - hs, cx + hs)
156         self._ax.set_ylim(cy - hs, cy + hs)
157         self._ax.set_aspect("equal", "box")
158
159         self._ax.set_xlabel("x (m)")
160         self._ax.set_ylabel("y (m)")
161         self._ax.legend(fontsize=8, loc="upper right")
162         self._toolbar.update()
163
164         self._texto_info = self._ax.text(
165             0.02, 0.95, "",
166             transform=self._ax.transAxes,
167             fontsize=9, verticalalignment="top",
168             bbox=dict(boxstyle="round", facecolor="white", alpha=0.85),
169         )
170         self._actualizar_marcaador(0)
171
172     def _actualizar_marcaador(self, paso: int):
173         self._label_paso.setText(f"Paso: {paso} / {self._slider.maximum()}")
174         lineas = []
175
176         for df_orion, df_luna, marc_orion, marc_luna in self._marcadores:
177             idx = min(paso, len(df_orion) - 1)
178
179             marc_orion.set_data([df_orion["x"].iloc[idx]],
180                                [df_orion["y"].iloc[idx]])
181             if df_luna is not None and marc_luna is not None:
182                 idx_l = min(paso, len(df_luna) - 1)
183                 marc_luna.set_data([df_luna["x"].iloc[idx_l]],
184                                    [df_luna["y"].iloc[idx_l]])
185
186             vx = df_orion["vx"].iloc[idx]
187             vy = df_orion["vy"].iloc[idx]
188             t_s = df_orion["time_s"].iloc[idx]
189
190             mag_v = np.hypot(vx, vy)
191             angulo_deg = np.degrees(np.arctan2(vy, vx))
192             fecha_str = (FECHA_INICIO + datetime.timedelta(seconds=float(t_s))
193                          ).strftime("%Y-%m-%d %H:%M:%S")
```

```
192         lineas.append(  
193             f"--- Orion ---\n"  
194             f"Fecha/Hora: {fecha_str}\n"  
195             f"Mag. Vel: {mag_v:.2f} m/s\n"  
196             f"Angulo Vel: {angulo_deg:.2f}  "  
197         )  
198  
199         if self._texto_info:  
200             self._texto_info.set_text("\n".join(lineas))  
201  
202         self._canvas.draw_idle()
```

7.5. Lectura/Escritura de archivos .csv

Con este código se crean los .csv resultado y se leen para realizar la simulación de la trayectoria de orión.

```
1  """  
2  Codigo hecho por: Agustin Dubovitsky Otero (padron: 111954) y Tomas Bautista Conti  
   (padron: 111760)  
3  Lectura y escritura de CSVs  
4  """  
5  
6  import os  
7  import sys  
8  
9  import numpy as np  
10 import pandas as pd  
11  
12  
13 def _normalizar_columnas(df: pd.DataFrame) -> pd.DataFrame:  
14     """Renombra columnas a nombres canonicos (time_s, x, y, vx, vy)."""  
15     col_map = {}  
16     for col in df.columns:  
17         cu = col.strip().upper()  
18         if cu in ("JDTDB", "TIME_S", "TIME"): col_map[col] = "time_s"  
19         elif cu in ("X", "X_LUNA"): col_map[col] = "x"  
20         elif cu in ("Y", "Y_LUNA"): col_map[col] = "y"  
21         elif cu == "VX": col_map[col] = "vx"  
22         elif cu == "VY": col_map[col] = "vy"  
23     return df.rename(columns=col_map)  
24  
25  
26 def _convertir_a_metros(df: pd.DataFrame) -> pd.DataFrame:  
27     """Convierte km a metros."""  
28     if df["x"].abs().max() < 1e7:  
29         df = df.copy()  
30         df["x"] *= 1_000  
31         df["y"] *= 1_000  
32     return df  
33  
34  
35 def _limpiar(df: pd.DataFrame) -> pd.DataFrame:  
36     for c in ("x", "y"):  
37         df[c] = pd.to_numeric(df[c], errors="coerce")  
38     return df.dropna(subset=["x", "y"])  
39  
40  
41 # Lectura  
42  
43 def cargar_csv_trayectoria(ruta: str) -> pd.DataFrame:  
44     """Lee un CSV de (propio o Horizons).  
45     Devuelve DataFrame con columnas [time_s, x, y, ...] en metros."""  
46     with open(ruta) as f:  
47         lineas = f.readlines()  
48     if any("; " in l for l in lineas[:5]):
```

```
49     df = pd.read_csv(ruta, sep=";", decimal=",", header=None,
50                      names=["time_s", "x", "y", "z", "vx", "vy", "vz"])
51     else:
52         # Buscar la fila de encabezado con columnas X e Y
53         header_idx = next(
54             (i for i, l in enumerate(lineas)
55              if {"X", "Y"} <= {p.strip().upper() for p in l.split(",")}),
56             None,
57         )
58         if header_idx is None:
59             print(f"Error: no se encontro encabezado con X e Y en '{ruta}'",
60                   file=sys.stderr)
61             sys.exit(1)
62         df = pd.read_csv(ruta, sep=",", header=header_idx, low_memory=False)
63
64     df = _normalizar_columnas(df)
65     df = _limpiar(df)
66     df = _convertir_a_metros(df)
67
68     print(f" [CSV] {os.path.basename(ruta)} cargado ({len(df):,} filas)")
69     return df
70
71
72 def cargar_posiciones_luna(ruta: str) -> tuple[np.ndarray, pd.DataFrame]:
73     """Carga un CSV de orbita lunar generado por trayectoria_luna.py.
74     Devuelve (array de posiciones, DataFrame completo)."""
75     df = cargar_csv_trayectoria(ruta)
76     return df[["x", "y"]].to_numpy(), df[["x", "y"]].reset_index(drop=True)
77
78
79 #         Escritura
80
81 def guardar_csv(df: pd.DataFrame, ruta: str) -> None:
82     """Guarda un DataFrame en CSV, creando la carpeta si no existe."""
83     os.makedirs(os.path.dirname(ruta) or ".", exist_ok=True)
84     df.to_csv(ruta, index=False)
85     print(f" Resultados guardados en: {ruta}")
```